

Plan maestro: App de fotos para eventos

Oct 6, 2026 · @Oscar

La mentalidad: un proyecto grande es muchas decisiones pequeñas bien escritas

Lo que separa un proyecto grande de uno pequeño no es la cantidad de código. Es que muchas personas (o tú dentro de seis meses) tienen que entenderlo, cambiarlo y desplegarlo sin miedo. Por eso este plan te entrena en cinco hábitos que usan los equipos profesionales:

1. **Diseñar antes de programar.** Requisitos, modelo de datos y contrato del API escritos antes de la primera línea de código.
2. **Documentar las decisiones, no solo el código.** Cada decisión importante queda en un ADR (Architecture Decision Record): qué se decidió, por qué y qué se descartó. Esto es lo que más hace un arquitecto.
3. **Avanzar en rebanadas verticales.** Cada entrega atraviesa frontend, backend, base de datos e infraestructura, aunque sea pequeña. Nunca "todo el backend primero".
4. **Producción desde la semana uno.** Un pipeline que despliega automáticamente a un entorno real desde el inicio. Los problemas de infraestructura se descubren temprano, cuando son baratos.
5. **La rama main siempre funciona.** Todo cambio entra por pull request, con pruebas y revisión.

La meta no es solo terminar la app. Es que al final puedas explicar en una entrevista cada decisión de arquitectura y sus alternativas.

Cómo trabajar con Claude Code sin dejar de aprender

La regla de oro: **tú decides y diseñas; Claude Code acelera y revisa.** Si Claude escribe todo, terminas con una app pero sin las habilidades. Este reparto equilibra velocidad y aprendizaje:

Tarea	Quién la hace	Por qué
Decisiones de arquitectura y ADRs	Tú (Claude propone opciones y contraargumentos)	Es exactamente la habilidad que quieres desarrollar
Modelo de datos y contrato del API	Tú primero, Claude revisa	Obliga a pensar en patrones de acceso
Código nuevo en React y NestJS	Tú escribes la primera versión de cada patrón	Es la única forma de aprender la tecnología
Código repetitivo (el segundo, tercer y décimo endpoint)	Claude Code	Ya entendiste el patrón
Infraestructura con Terraform	Juntos: Claude genera, tú lees línea por línea	Ya sabes Terraform; aquí el reto son los módulos y los entornos
Pruebas	Claude genera, tú decides qué casos importan	Aprendes a pensar en casos borde
Revisión de código	Claude revisa cada PR tuyo	Simula tener un senior en el equipo

Prácticas concretas:

- **Un archivo `CLAUDE.md` en la raíz del repo** con el stack, las convenciones y los comandos. Claude Code lo lee en cada sesión, así no repites contexto.
- **Antes de aceptar código de Claude, explícalo en voz alta.** Si no puedes explicar una línea, pregúntale antes de seguir.
- **Pídele "modo mentor" cuando aprendas algo nuevo:** que te dé pistas y revise tu intento en lugar de darte la solución.
- **Una tarea por sesión, ligada a un issue.** Sesiones cortas y enfocadas producen mejor código que sesiones largas y difusas.

Fase 0 · Diseño antes de código (1-2 semanas)

Entregable: una carpeta `docs/` en el repo con todo lo que un desarrollador nuevo necesitaría para entender el sistema sin hablar contigo.

- ☐ **Requisitos funcionales.** Historias de usuario por rol: invitado, dueño del evento, administrador. Ejemplo: "Como invitado, quiero subir hasta 10 fotos sin crear cuenta".
- ☐ **Requisitos no funcionales.** Cuántos invitados por evento, tamaño máximo por foto, cuántos días se guardan las fotos, presupuesto mensual objetivo.
- ☐ **Diagrama de arquitectura** (formato C4, niveles 1 y 2): contexto y contenedores. Herramienta sugerida: draw.io o Excalidraw.
- ☐ **ADR-001 a ADR-007:** React + Vite, NestJS, Aurora Serverless v2 (PostgreSQL), Lambda en vez de ECS, Cognito, Terraform, un solo repositorio (monorepo). Plantilla corta: contexto, decisión, alternativas, consecuencias.
- ☐ **Modelo de datos en PostgreSQL.** Diagrama entidad-relación (eventos, fotos, dispositivos, usuarios) con llaves, índices y restricciones. Por ejemplo, una fila única por evento y dispositivo para llevar el contador de fotos.
- ☐ **Contrato del API en OpenAPI.** Endpoints, cuerpos y códigos de error, antes de implementarlos.
- ☐ **Modelo de amenazas básico.** ¿Qué pasa si alguien sube 10.000 fotos, archivos que no son imágenes o contenido inapropiado?

Esta fase se siente lenta, pero es la que más se parece al trabajo de un arquitecto. Usa a Claude como crítico: "¿qué falla en este diseño cuando un evento tiene 500 invitados?"

Fase 1 · Cimientos (1-2 semanas)

Entregable: un "hola mundo" de punta a punta. Una página de React en CloudFront que llama a un endpoint de NestJS en Lambda, desplegado automáticamente al hacer merge en main.

- ☐ **Un solo repositorio con carpetas:** `apps/web` (React), `apps/api` (NestJS), `packages/shared` (tipos compartidos) e `infra/` (Terraform).
- ☐ **pnpm workspaces** para que `web` y `api` importen `packages/shared` como si fuera un paquete npm, sin publicarlo.
- ☐ **Tipos generados desde el contrato:** el OpenAPI vive en `apps/api` y `openapi-typescript` genera los tipos en `packages/shared`. Si cambia un endpoint, el frontend deja de compilar hasta que lo actualices.
- ☐ **Calidad automática:** ESLint, Prettier, TypeScript en modo estricto y commits convencionales, configurados una sola vez en la raíz.

- ☐ **Terraform con módulos reutilizables** (`frontend` , `api` , `auth` , `data`) y una carpeta por entorno (`infra/envs/dev` , `infra/envs/prod`) que llama a los mismos módulos con variables distintas.
- ☐ **Estado remoto en S3** con bloqueo nativo (`use_lockfile`), un estado por entorno.
- ☐ **Dos entornos en cuentas separadas** con AWS Organizations (no tiene costo extra). Servicios caros como WAF solo en `prod` .
- ☐ **CI/CD con GitHub Actions y filtros por carpeta (`paths:`)**: un cambio en `apps/web` solo despliega el frontend. `terraform plan` comentado en cada PR que toque `infra/` ; `apply` a `dev` al hacer merge; `prod` con aprobación manual usando GitHub Environments.
- ☐ **Autenticación de GitHub hacia AWS con OIDC**, sin llaves de acceso guardadas en secretos.
- ☐ **Alarma de presupuesto en AWS Budgets** en cada cuenta (por ejemplo, aviso a los \$10).
- ☐ **`CLAUDE.md` y `README.md`** en la raíz con la estructura del repo y cómo levantar el proyecto localmente.
- ☐ **Aurora Serverless v2 (PostgreSQL) al mínimo**: capacidad de 0 a 1 ACU, pausa tras 5–10 minutos sin actividad, configuración Standard, una sola instancia sin réplicas.
- ☐ **Acceso por Data API (HTTP)**: las Lambdas quedan fuera de la VPC, sin NAT Gateway ni pool de conexiones. Credenciales en Secrets Manager.
- ☐ **Migraciones versionadas** del esquema (por ejemplo, con Drizzle o Prisma), ejecutadas en el pipeline antes de desplegar el API.

Fases 2 a 5 · Construir en rebanadas verticales

Cada fase termina con algo que funciona en producción y que se podría mostrar a un cliente real.

Fase 2 · MVP del invitado (2–3 semanas)

Entregable: un evento creado a mano en la base de datos, un QR impreso y un amigo que sube fotos desde su teléfono.

- ☐ Página pública del evento por slug (`/e/boda-ana-luis`)
- ☐ Identificador de dispositivo (UUID en cookie) y contador de 10 fotos con un UPDATE condicional en PostgreSQL (atómico, sin condiciones de carrera)
- ☐ Endpoint que genera URLs prefirmadas de S3 con límite de tamaño y tipo de archivo

- ☐ Subida directa del navegador a S3, con barra de progreso y opción de tomar foto con la cámara
- ☐ Galería pública del evento
- ☐ Generación del código QR
- ☐ Despertar programado: EventBridge Scheduler ejecuta una consulta 30 minutos antes del inicio de cada evento, para que Aurora no arranque en frío con el primer invitado

Fase 3 · Usuarios y administración (2-3 semanas)

Entregable: un administrador crea un evento para un correo; el cliente inicia sesión con Google y ve sus fotos.

- ☐ User Pool de Cognito con login administrado, Google y correo/contraseña
- ☐ Grupo `admins` y guardas en NestJS por rol
- ☐ Panel de admin: crear, editar y desactivar eventos
- ☐ Vista "Mis eventos" para el dueño
- ☐ Descarga de todas las fotos en ZIP

Fase 4 · Procesamiento y experiencia (2 semanas)

Entregable: la galería carga rápido aunque el evento tenga cientos de fotos.

- ☐ Lambda disparada por S3 que genera miniaturas WebP con `sharp`
- ☐ Galería con carga diferida y vista ampliada
- ☐ Moderación: el dueño puede ocultar o borrar fotos
- ☐ Validación del contenido real del archivo (no solo la extensión)

Fase 5 · Listo para producción (1-2 semanas)

Entregable: podrías cobrarle a un cliente con tranquilidad.

- ☐ Observabilidad: logs estructurados, métricas y alarmas en CloudWatch, trazas con X-Ray
- ☐ AWS WAF con rate limiting en el API
- ☐ Ciclo de vida en S3: borrar o archivar fotos N días después del evento
- ☐ Dominio propio y HTTPS con Route 53 y ACM
- ☐ Prueba de carga simulando 300 invitados subiendo a la vez
- ☐ Un runbook: qué hacer si algo falla durante un evento en vivo

Hábitos en cada fase y qué estudiar

Trabaja como si fueras un equipo, aunque seas uno solo:

- **GitHub Projects como tablero.** Cada tarea de este plan es un issue; cada issue, una rama y un PR.
- **Definición de "terminado":** el código tiene pruebas, pasó CI, está desplegado en `dev` y la documentación está actualizada.
- **Un ADR cada vez que dudes entre dos opciones.** Aunque sea de cinco líneas.
- **Retrospectiva al cerrar cada fase:** qué salió bien, qué no, qué cambias. Media página en `docs/retros/`.
- **Escribe en público.** Un post corto por fase (LinkedIn o un blog) sobre una decisión que tomaste. Construye tu marca profesional y te obliga a entender lo que hiciste.

Lecturas, en el orden en que te van a servir:

Cuándo	Recurso	Para qué
Fase 0	<i>Fundamentals of Software Architecture</i> (Richards y Ford)	Vocabulario de arquitecto: estilos, trade-offs, ADRs
Fase 0-2	<i>The Art of PostgreSQL</i> (Dimitri Fontaine)	SQL más allá de lo básico: índices, consultas y modelado
Fase 1-2	Documentación oficial de React y NestJS	La fuente más actualizada; mejor que cursos
Fase 1	Documentación de Terraform (módulos y estado remoto)	Infraestructura como código
Al terminar	<i>Designing Data-Intensive Applications</i> (Kleppmann)	El libro de referencia para sistemas distribuidos

Esta semana: cinco primeros pasos

- ☐ Ponerle nombre al proyecto y crear el repositorio en GitHub con la carpeta `docs/`
- ☐ Escribir tú las historias de usuario de los tres roles (sin Claude primero; después pídele que encuentre huecos)
- ☐ Escribir los requisitos no funcionales con números, aunque sean estimados
- ☐ Redactar ADR-001 (React + Vite) para practicar el formato

- ☐ Listar todos los patrones de acceso a datos que se te ocurran; con eso diseñamos juntos el esquema de PostgreSQL

Cuando termines, trae tus documentos a una sesión y los revisamos como lo haría un arquitecto senior.